1 Projet : VNF Chain Optimization

Ce notebook formalise la logique et la modélisation mathématique pour résoudre le projet d'optimisation (placements de VNF sur des serveurs).

1.1 Objectif Global

Trouver un placement optimal de n fonctions réseau virtuelles (VNF) sur m serveurs physiques qui minimise la charge maximale parmi tous les serveurs (le goulot d'étranglement ou **cycle time** CT), tout en respectant les contraintes de précédence entre les VNF.

---

2 Phase 1 : Solution Exacte (MILP)

Cette phase formalise le problème comme un programme linéaire en nombres entiers (MILP) et utilise Gurobi pour trouver la solution optimale.

2.1 1. Formulation Mathématique

2.1.1 1.1 Variables de Décision

- $x_{jk} \in \{0, 1\}$: Vaut 1 si la VNF j est placée sur le serveur k, 0 sinon.
- $s_j \in \{1, \ldots, m\}$: Indice du serveur sur lequel la VNF j est placée.
- $CT \geq 0$: Le "Cycle Time" ou latence du goulot d'étranglement (charge maximale d'un serveur).

2.1.2 1.2 Fonction Objectif

Minimiser CT

2.1.3 1.3 Contraintes

1. **Unicité du placement** : Chaque VNF j doit être placée sur exactement un serveur.
$$\sum_{k=1}^{m} x_{jk} = 1 \quad \forall j \in J$$
2. **Définition de l'indice du serveur** : Lien entre les variables binaires x et l'indice s_j.
$$s_j = \sum_{k=1}^{m} k \cdot x_{jk} \quad \forall j \in J$$
3. **Respect des précédences** : Pour chaque arc $(i, j) \in P$, la VNF i doit être placée sur le même serveur ou un serveur précédent par rapport à j.
$$s_i \leq s_j \quad \forall (i, j) \in P$$

4. **Goulot d'étranglement** : La somme des temps de traitement sur n'importe quel serveur k ne doit pas dépasser le cycle time.

$$\sum_{j=1}^{n} t_j \cdot x_{jk} \leq CT \quad \forall k \in K$$

2.2 2. Résultats et Analyse

2.2.1 2.1 Tableau des résultats

Le tableau suivant regroupe les résultats de notre modèle MILP sur les 24 instances du benchmark SALBPGen. Le nombre de serveurs m est spécifique à chaque instance (issu du benchmark). Une limite de temps de 30 minutes par instance a été imposée.

Instance	$\boldsymbol{n}$	$\boldsymbol{m}$	**OS**	**Préc.**	$\sum t_j$	$\boldsymbol{CT^*}$	**Temps (s)**
n=20_1	20	3	0.268	16	2882	962	0.054
n=20_2	20	3	0.300	19	2861	954	0.007
n=20_3	20	3	0.300	18	2785	930	0.056
n=20_4	20	3	0.289	18	2727	909	0.036
n=20_5	20	3	0.289	18	2837	950	0.076
n=20_6	20	3	0.295	19	2914	973	0.045
n=20_7	20	3	0.300	18	2786	930	0.041
n=20_8	20	3	0.289	20	2985	996	0.071
n=20_9	20	3	0.289	18	2925	976	0.055
n=20_10	20	3	0.279	17	2831	945	0.067
n=20_11	20	3	0.279	18	2766	924	0.089
n=20_12	20	3	0.300	17	2930	977	0.023
n=20_13	20	3	0.268	17	2896	967	0.078
n=20_14	20	3	0.263	17	2979	994	0.086
n=20_15	20	3	0.295	18	2981	994	0.037
n=20_16	20	12	0.232	17	10376	981	0.751
n=20_17	20	10	0.295	17	9134	969	0.366
n=20_18	20	11	0.300	20	9524	960	0.191
n=20_19	20	14	0.268	18	11113	1000	0.240
n=20_20	20	11	0.295	17	9829	1000	0.298
n=50_1	50	8	0.196	58	7276	910	11.52
n=50_2	50	6	0.195	56	5585	931	1.49
n=100_1	100	23	0.196	105	22723	N/A†	>1800
n=100_2	100	21	0.195	104	20262	N/A†	>89

Table 1 – Résultats complets de l'optimisation MILP (Phase 1). †Instances n'ayant pas convergé dans la limite de 30 minutes.

2.2.2 2.2 Analyse graphique des performances

La figure 1 illustre l'évolution du temps de résolution en fonction de la taille de l'instance et de l'Order Strength.

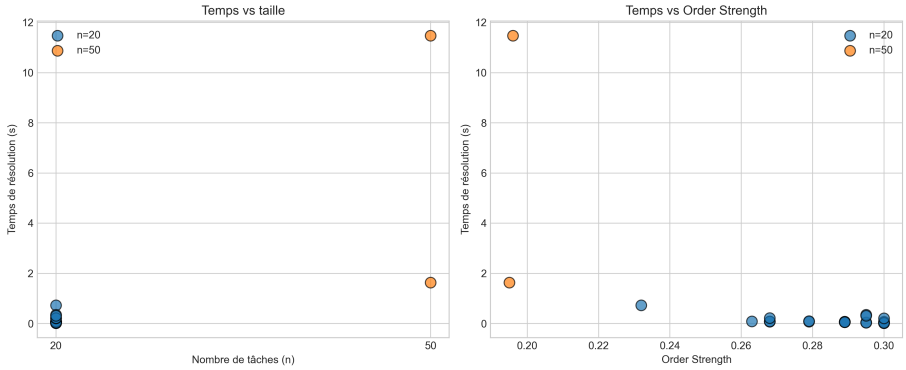

FIGURE 1 – Temps de résolution en fonction de la taille du problème (n) et de l'Order Strength (OS).

2.3 Conclusions

- **Instances** $n = 20$: Toutes les 20 instances sont résolues à l'optimalité en moins d'une seconde (≤ 0.75s). Les CT^* obtenus respectent la borne $CT \leq 1000$ imposée par le benchmark. Les instances 1–15 ($m = 3$) sont les plus rapides ; les instances 16–20 ($m \geq 10$), plus coûteuses en nombre de variables binaires, restent néanmoins résolues très efficacement.
- **Instances** $n = 50$: Résolues à l'optimalité en 1.5s à 11.5s. L'augmentation significative du temps pour n=50_1 ($m = 8$) par rapport à n=50_2 ($m = 6$) illustre l'impact du nombre de serveurs sur la combinatoire ($n \times m$ variables binaires).
- **Instances** $n = 100$: Non résolues dans la limite de 30 minutes. Avec $m \geq 21$ serveurs, le modèle comporte plus de $n \times m = 2100$ variables binaires, rendant l'exploration du Branch-and-Bound très coûteuse. Cela motive le recours aux heuristiques (Phase 2).
- **Impact de** m **vs.** n : Le facteur déterminant pour la difficulté n'est pas uniquement n mais le produit $n \times m$, qui détermine le nombre de variables binaires x_{jk}.

3 Phase 2 : Heuristiques et Métaheuristiques

3.1 Motivation

Les solveurs MILP passent difficilement à l'échelle au-delà de quelques centaines de variables binaires pour des instances complexes. Comme observé en Phase 1, les instances $n = 100$ n'ont pas convergé en 30 minutes, et le temps de résolution augmente fortement avec le produit $n \times m$. La Phase 2 propose trois méthodes approximatives capables de produire de bonnes solutions en une fraction du temps du solveur exact.

3.2 Description des méthodes

Trois méthodes ont été implémentées, couvrant le spectre des approches classiques de la recherche opérationnelle : une heuristique constructive déterministe, et deux métaheuristiques (SA et GA) dotées de mécanismes d'exploration stochastique.

3.2.1 H1 — Heuristique gloutonne constructive (*Greedy*)

Principe. L'heuristique gloutonne construit une solution en une seule passe, sans retour-arrière. Les VNF sont traitées dans l'ordre topologique du DAG de précédences (algorithme de Kahn), ce qui garantit que chaque VNF est affectée *après* tous ses prédécesseurs.

Pour chaque VNF j traitée dans cet ordre, on détermine l'ensemble des serveurs candidats :

$$C_j = \left\{ k \in K \mid k \geq \max_{(i,j) \in P} s_i \right\}$$

puis on affecte j au serveur qui minimise l'augmentation de charge :

$$s_j = \arg\min_{k \in C_j} \left(L_k + t_j \right).$$

Justification. Le critère d'affectation greedy (minimiser la charge du serveur cible) est l'heuristique de référence pour les problèmes d'équilibrage de charge. Couplée au tri topologique, elle garantit la faisabilité à chaque étape sans mécanisme de réparation. Sa complexité est $O(n \cdot m)$, ce qui la rend quasi-instantanée.

Paramètres. Aucun hyperparamètre. Solution entièrement déterministe.

3.2.2 H2 — Recuit Simulé (*Simulated Annealing*, SA)

Principe. Le recuit simulé est une métaheuristique de trajectoire qui explore l'espace des solutions en acceptant, avec une probabilité décroissante, des solutions légèrement dégradantes. L'algorithme part de la solution greedy, puis à chaque itération :

1. génère un voisin en déplaçant une VNF j choisie aléatoirement vers un serveur k' aléatoire ;
2. répare les violations de précédence induites par propagation (on force $s_{j'} \geq s_i$ pour tout successeur j' concerné) ;
3. accepte le voisin si $\Delta CT < 0$, ou avec probabilité $\exp(-\Delta CT/T)$ sinon.

La température T décroît géométriquement : $T \leftarrow \alpha \cdot T$.

Justification. Le SA est particulièrement adapté aux espaces de solutions fortement contraints. La stratégie de réparation par propagation (plutôt qu'un rejet systématique) permet une exploration plus riche de l'espace faisable tout en maintenant la faisabilité.

Paramètre	**Valeur**	**Justification**
T_{init}	500	Acceptation initiale de dégradations d'environ 5%
$T_{\min}$	0.1	En-dessous, $\exp(-\Delta/T) \approx 0$
α	0.995	Refroidissement lent, $\sim$9 000 pas avant $T < 1$
`max_iter`	50 000	Temps raisonnable sur toutes tailles d'instance
`seed`	42	Reproductibilité des résultats

Table 2 – Hyperparamètres du recuit simulé (H2).

Hyperparamètres et réglage.

3.2.3 H3 — Algorithme Génétique (*Genetic Algorithm*, GA)

Principe. L'algorithme génétique maintient une *population* de solutions et la fait évoluer par sélection, croisement et mutation. À chaque génération :

1. **Élitisme :** les e meilleures solutions sont conservées sans modification.
2. **Sélection par tournoi :** on tire τ individus aléatoirement et on retient le meilleur comme parent.
3. **Croisement uniforme :** pour chaque VNF j, l'enfant hérite du parent 1 ou du parent 2 avec probabilité $\frac{1}{2}$. Les violations de précédence sont réparées par un balayage topologique.
4. **Mutation :** chaque VNF est réaffectée aléatoirement avec probabilité p_m, puis les précédences sont réparées.

La population initiale est générée par des heuristiques greedy avec ordre topologique perturbé (shuffle + retri topologique), complétée par la solution greedy pure comme amorce de qualité.

Justification. La gestion d'une population permet d'explorer plusieurs bassins d'attraction simultanément, là où le SA reste sur une seule trajectoire. Le croisement uniforme avec réparation est adapté aux problèmes d'affectation contrainte car il combine les décisions des deux parents VNF par VNF.

Paramètre	Valeur	Justification
`pop_size`	60	Compromis diversité / temps de calcul
`n_generations`	300	Convergence observée avant 300 générations
p_m (taux de mutation)	0.05	5% par gène, évite la convergence prématurée
`tournament_size`	5	Pression sélective modérée
`elite_size`	4	Préserve les meilleures solutions
`seed`	42	Reproductibilité

TABLE 3 – Hyperparamètres de l'algorithme génétique (H3).

Hyperparamètres et réglage.

3.3 Résultats

3.3.1 Tableau comparatif

Le tableau 4 présente, pour chaque instance et chaque heuristique, le cycle time obtenu (CT), le gap d'optimalité et le temps CPU. Le gap est :

$$\text{Gap} = \frac{CT_{\text{heuristique}} - CT^*}{CT^*} \times 100\%.$$

Pour les instances $n = 100$, aucune valeur CT^* n'étant disponible (Phase 1 non convergée), le gap n'est pas calculé.

3.3.2 Résumé statistique

3.4 Analyse critique

3.4.1 Aucune méthode n'atteint l'optimum exact

Les résultats montrent qu'aucune des trois méthodes n'atteint le CT^* prouvé sur les 22 instances pour lesquelles il est disponible. Cela s'explique principalement par la structure du problème : avec $m = 3$ serveurs et des précédences denses (OS ≈ 0.28), les solutions sous-optimales du greedy sont extrêmement sous-optimales (gaps $> 100\%$), et les métaheuristiques convergent vers des minima locaux proches mais non exactement égaux à l'optimum.

3.4.2 Analyse par méthode

H1 — Greedy (gap moyen : 151%). Le greedy est catastrophique sur les instances $m = 3$ car son critère local — minimiser la charge courante du serveur cible — ignore les contraintes globales de précédence. Avec un ordre topologique fixe, il entasse souvent toutes les VNF lourdes sur le premier serveur disponible, produisant des déséquilibres massifs (gap jusqu'à 321% sur n=20_17). Il est en revanche quasi-instantané (<0.001 s).

Instance	n	m	CT^*	H1 — Greedy			H2 — SA			H3 — GA		
				CT	Gap%	t(s)	CT	Gap%	t(s)	CT	Gap%	t(s)
n=20_1	20	3	962	1311	36.3	<0.001	980	1.9	0.010	986	2.5	1.32
n=20_2	20	3	954	2579	170.3	<0.001	961	0.7	0.010	966	1.3	1.38
n=20_3	20	3	930	2211	137.7	<0.001	936	0.6	0.010	939	1.0	1.36
n=20_4	20	3	909	2185	140.4	<0.001	942	3.6	0.010	913	0.4	1.35
n=20_5	20	3	950	2499	163.1	<0.001	963	1.4	0.010	963	1.4	1.37
n=20_6	20	3	973	1942	99.6	<0.001	982	0.9	0.010	978	0.5	1.39
n=20_7	20	3	930	1541	65.7	<0.001	947	1.8	0.010	937	0.8	1.35
n=20_8	20	3	996	2724	173.5	<0.001	1005	0.9	0.010	999	0.3	1.40
n=20_9	20	3	976	1886	93.2	<0.001	1001	2.6	0.010	982	0.6	1.37
n=20_10	20	3	945	2187	131.4	<0.001	971	2.8	0.010	964	2.0	1.34
n=20_11	20	3	924	2184	136.4	<0.001	932	0.9	0.010	926	0.2	1.35
n=20_12	20	3	977	2232	128.5	<0.001	995	1.8	0.010	987	1.0	1.35
n=20_13	20	3	967	2212	128.8	<0.001	982	1.6	0.010	969	0.2	1.34
n=20_14	20	3	994	2315	132.9	<0.001	1010	1.6	0.010	1005	1.1	1.33
n=20_15	20	3	994	2535	155.0	<0.001	1036	4.2	0.010	1012	1.8	1.35
n=20_16	20	12	981	2844	189.9	<0.001	1088	10.9	0.011	1062	8.3	1.45
n=20_17	20	10	969	4083	321.4	<0.001	1133	16.9	0.011	1128	16.4	1.42
n=20_18	20	11	960	1953	103.4	<0.001	1055	9.9	0.012	1023	6.6	1.52
n=20_19	20	14	1000	2319	131.9	<0.001	1110	11.0	0.011	1110	11.0	1.49
n=20_20	20	11	1000	2096	109.6	<0.001	1035	3.5	0.011	1047	4.7	1.52
n=50_1	50	8	910	5047	454.6	<0.001	1116	22.6	0.024	1019	12.0	4.90
n=50_2	50	6	931	2047	119.9	<0.001	955	2.6	0.025	938	0.8	4.72
n=100_1	100	23	—	10557	—	<0.001	2753	—	0.048	2068	—	13.90
n=100_2	100	21	—	11818	—	<0.001	3321	—	0.046	1533	—	13.90

Table 4 – Comparatif des trois heuristiques sur les 24 instances. Gap en %, temps CPU en secondes. Les instances $n = 100$ n'ont pas de CT^* prouvé (Phase 1 non convergée).

Méthode	Gap moyen (%)	Instances à l'optimal	Temps moyen (s)
H1 — Greedy	151.1	0 / 22	<0.001
H2 — SA	4.8	0 / 22	0.014
H3 — GA	3.4	0 / 22	2.715

Table 5 – Résumé par heuristique (instances $n \leq 50$, CT^* connu). Aucune méthode n'atteint l'optimum exact ; le GA obtient le meilleur gap moyen au prix d'un temps de calcul environ 200 fois supérieur au SA.

H2 — SA (gap moyen : 4.8%, temps moyen : 0.014 s). Le SA apporte une amélioration très significative par rapport au greedy, avec un gap moyen de 4.8% en moins de 15 ms. Sur les instances $m = 3$ ($n = 20$, instances 1–15), le gap reste inférieur à 5%, ce qui illustre l'efficacité de l'exploration par perturbation locale. Les instances à m élevé (16–20) et $n = 50_1$ sont plus difficiles (gaps 10–23%), car l'espace de recherche est plus vaste et le SA, limité à une seule trajectoire, peine à en explorer la totalité dans 50 000 itérations.

H3 — GA (gap moyen : 3.4%, temps moyen : 2.7 s). Le GA obtient le meilleur gap moyen (3.4%), notamment sur les grandes instances : sur n=50_1, il descend à 12% là où le SA plafonne à 23% ; sur n=100_2, il produit CT=1533

contre CT=3321 pour le SA, soit un écart de plus du double. La diversité de population permet d'explorer plusieurs bassins d'attraction simultanément. En contrepartie, son temps de calcul (2.7 s en moyenne, jusqu'à 15 s sur $n = 100$) est environ 200 fois supérieur à celui du SA.

3.4.3 Propriétés structurelles facilitant la résolution

Les instances les plus faciles pour les métaheuristiques présentent deux caractéristiques :

— **OS élevé et m faible ($m = 3$, instances 1–15).** Les précédences denses contraignent fortement l'espace faisable. Le SA et le GA convergent rapidement car peu de réaffectations sont possibles ; les gaps restent inférieurs à 5%.

— **Faible rapport n/m avec m petit.** Sur n=50_2 ($m = 6$), le GA atteint un gap de 0.75%, très proche de l'optimum, car la charge par serveur est élevée et les mouvements de VNF ont un impact limité sur CT.

À l'inverse, les instances difficiles (n=20_17 avec $m = 10$, n=50_1 avec $m = 8$) combinent un grand nombre de serveurs et un OS faible : l'espace faisable est vaste, les solutions équilibrées sont rares, et les deux métaheuristiques convergent vers des minima locaux éloignés de l'optimum.

3.4.4 Limites

Sur les instances $n = 100$ ($m \geq 21$), aucun CT^* n'est disponible pour quantifier les gaps. Les écarts considérables entre SA et GA (factor 2 sur n=100_2) suggèrent que ni l'une ni l'autre ne trouve une solution proche de l'optimum. Une stratégie hybride — initialisation par GA suivie d'une intensification MILP sur un sous-problème restreint — constituerait une piste d'amélioration naturelle.